

Ejercicio de Laboratorio (17/02/2026)

Fundamentos de la Programación II

Grupos DG-MAT-INF, A

Un **polinomio** es una expresión algebraica formada por la suma de un número finito de términos llamados **monomios**. Los monomios son una expresión algebraica formada por un *coeficiente*, una variable y una *potencia* de exponente natural. En general, los polinomios no se encuentran definidos en los lenguajes de programación, por ello es necesario usar estructuras para representarlos. Por ejemplo, podemos usar una lista para representar polinomios de cualquier grado sobre una variable x , siendo el grado el mayor exponente de los monomios que lo forman. Por ejemplo, el siguiente polinomio de grado n :

$$a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$$

se puede representar mediante una lista ordenada de forma decreciente con respecto al grado n del polinomio:

$$(a_n, n), (a_{n-1}, n-1), \dots, (a_0, 0)$$

Cada elemento de la lista representa un monomio (coeficiente, grado). Por ejemplo, el polinomio $p(x) = 3x^4 + 2$, se representa como:

$$(3, 4), (2, 0)$$

Observa que los monomios con coeficiente 0 no aparecen en la lista y ten en cuenta que un polinomio nulo se representa con la lista vacía.

El ejercicio consistirá en implementar una aplicación que permita realizar la suma y el producto de polinomios. Para ello necesitarás definir dos clases: `tMonomio` y `tPolinomio`. Recuerda que para cada clase necesitas añadir un fichero `.h` y un fichero `.cpp`.

Clase `tMonomio`

Encapsula la información de un monomio. Para ello necesitas almacenar información sobre el grado y el coeficiente (ambos de tipo entero) del monomio. Define el módulo `monomio`, donde `monomio.h` tendrá la siguiente forma:

```
#pragma once

class tMonomio {
private:
    int grado;
    int coef;
public:
    tMonomio();
    void inicia(int c, int g);
    bool operator<(const tMonomio& m) const;
    tMonomio operator+(const tMonomio& m2) const;
```

```
tMonomio operator*(const tMonomio& m2) const;
bool esMonomioNulo() const;
int dameGrado() const;
int dameCoeficiente() const;
};
```

Donde las funciones hacen lo siguiente:

- `monomio()`: constructora por defecto. Pone el coeficiente y el grado a 0.
- `inicia(c,g)`: inicia el monomio con coeficiente `c` y grado `g`.
- `operator<(m)`: devuelve `true` si y sólo si el grado de `this` es menor que el grado de `m`.
- `operator+(m2)`: devuelve la suma de los monomios `this` y `m2`. A este operador siempre se le llamará con monomios del mismo grado.
- `operator*(m2)`: devuelve el producto de los monomios `this` y `m2`. Se multiplican coeficientes y se suman grados.
- `monomioNulo()`: devuelve `true` si y solo si el coeficiente del monomio `this` es 0.
- `dameGrado()`: devuelve el grado del monomio `this`.
- `dameCoeficiente()`: devuelve el coeficiente del monomio `this`.

Clase tPolinomio

Encapsula la información de un polinomio, es decir una lista de monomios de tamaño máximo `MAX_MONOMIOS`. No olvides que en la representación de un polinomio no se admiten monomios cuyo grado sea 0 y que, además, el polinomio almacena los monomios ordenados por el grado, de mayor a menor. No tienes que realizar ningún tipo de ordenación en este ejercicio, ya que los polinomios se leerán de archivos que contendrán los monomios de forma ordenada. Si no fuera así, la aplicación lo detectará y terminará con un mensaje de error. Te mostramos un ejemplo de archivo:

```
5
3 6
-3 5
2 3
7 2
4 0
```

donde la primera línea (5) indica el número de monomios del polinomio. A continuación cada línea es un monomio, donde el primer entero es el coeficiente y el segundo entero es el grado.

Crea el módulo `polinomio`, donde `polinomio.h` tendrá la siguiente estructura:

```
#pragma once

#include "monomio.h"
#include <fstream>
```

```

using namespace std;
const int MAX_GRADO = 20;

class tPolinomio {
private:
    int cont;
    tMonomio monomios[MAX_GRADO];
    // declara métodos privados si es necesario

public:
    tPolinomio();
    void inicia();
    bool insertar(const tMonomio& m);
    tPolinomio operator+(const tPolinomio& p2) const;
    int dameLongitud() const;
    tMonomio dameElem(int i) const;
    tPolinomio operator*(const tPolinomio& p2) const;
};

```

Las operaciones públicas que ofrece la clase, y que debes implementar son las siguientes:

- `tPolinomio()`: constructora por defecto. Inicia el número de elementos de la lista a 0.
- `inicia()`: similar a la constructora por defecto.
- `insertar(m)`: inserta el monomio `m` al final de la lista comprobando que el grado de dicho monomio es menor que el grado del último monomio almacenado en la lista. Si no fuera así devuelve `false`. En otro caso devuelve `true`. De esta forma se garantiza que la lista siempre estará ordenada de mayor a menor por los grados de los monomios.
- `dameLongitud()`: devuelve la longitud de la lista.
- `dameElem(i)`: devuelve el elemento `i`-ésimo de la lista.
- `operator+(p2)`: devuelve la suma del objeto `this` y `p2`. Ten en cuenta que los polinomios tienen los monomios ordenados por el grado. Para implementar este operador necesitarás utilizar el operador `+` de la clase `tMonomio` y tener cuidado de no almacenar en el polinomio resultado monomios nulos. Por ejemplo, la suma de los polinomios:

$$\begin{array}{r} x^4 + 2x \\ 4x^3 - 2x + 3 \end{array}$$

devolverá el polinomio $x^4 + 4x^3 + 3$. El monomio correspondiente a grado 1 desaparece ya que su coeficiente es 0. Para implementar la suma basta recorrer ambos polinomios a la vez. El monomio de grado mayor en el recorrido se guarda en el polinomio resultado, avanzando sólo en ese polinomio. Cuando los monomios tienen el mismo grado entonces la suma de monomios debe ejecutarse y el resultado almacenarse en el polinomio resultado si el coeficiente no es 0. Una vez acabado un polinomio, el otro se recorre completamente y se almacena en el polinomio resultado. Si es necesario, implementa métodos privados.

- `operator*(p2)`: devuelve el producto de los polinomios `this` y `p2`. En este caso es suficiente recorrer `this`, y multiplicar cada monomio de `this` por `p2`, almacenándolo con cuidado en el polinomio resultado usando la suma de polinomios. Para implementar este operador, implementa previamente el operador privado `tPolinomio operator*(const tMonomio& m) const`; que devuelve la multiplicación del polinomio `this` por el monomio `m`.

Módulo operadores

En este módulo vamos a implementar los operadores de lectura/escritura necesarios para realizar el ejercicio. Concretamente `operadores.h` tendrá la siguiente forma:

```
#pragma once
#include <fstream>
#include <iostream>
#include "polinomio.h"

using namespace std;

istream& operator>>(istream& input, tMonomio& mon);
ostream& operator<<(ostream& out, const tPolinomio& pol);
```

Debes implementar estos operadores en `operadores.cpp`, de forma que la salida sea similar a la que te mostraremos más adelante.

Función main

Esta función lee dos polinomios, utilizando alguno de los archivos de texto que se adjuntan con el enunciado. Si la lectura ha sido correcta, muestra la suma y el producto de ambos polinomios.

Te mostramos algunos ejemplos de ejecución:

Para los archivos `polinomios1.txt` y `polinomios3.txt` obtenemos la salida:

```
cargando primer polinomio....
Polinomio cargado correctamente
+ 2x4 + 3x2 + 5
cargando segundo polinomio....
El polinomio contenido en el archivo no tiene el formato correcto
```

Para los archivos `polinomios1.txt` y `polinomios2.txt` obtenemos la salida:

```
cargando primer polinomio....
Polinomio cargado correctamente
+ 2x4 + 3x2 + 5
cargando segundo polinomio....
Polinomio cargado correctamente
+ 3x6 + 3x2
```

La suma de los polinomios es: + 3x6 + 2x4 + 6x2 + 5

El producto de los polinomios es: + 6x10 + 9x8 + 21x6 + 9x4 + 15x2